

Computer Hardware

Threads and Synchronisation in C++

Ksawery Moźdzynski

Agenda For Today

 1. Threads

`std::thread, join`: three workers and one fridge

 2. Race condition

When the result depends on the timing

 3. Mutex

`lock(), unlock(), lock_guard`

 4. Atomic operations

`std::atomic`: one step that cannot be split

 5. Deadlock

Two locks, and everybody waits forever

 6. Fixing a deadlock

One order for everyone, `scoped_lock`

 7. Condition variables

Sleep until somebody calls you

 8. Producer and consumer

Two kinds of waiting, one warehouse

 9. Semaphores

A counter with waiting built in

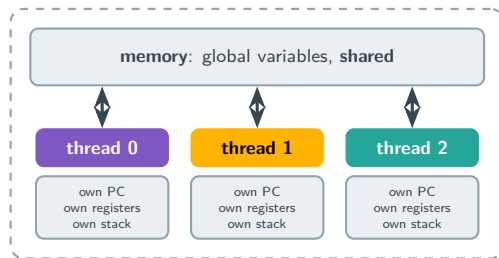
What is a thread?

A **thread** is an independent sequence of instructions inside a program.

Each thread has its own **PC**, **registers** and **stack**.

All threads of a program share the **same memory** (global variables).

one program



R. H. Arpaci-Dusseau, A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*

Threads in C++

```
● ● ●  
  
#include <iostream>  
#include <thread>  
  
void hello() {  
    std::cout << "Hello\n";  
}  
  
int main() {  
    std::thread t(hello);  
    t.join();  
}
```

The whole program

main starts a second thread. It prints Hello, and main waits for it.

Two new things

std::thread and join. One by one.

Threads in C++: `std::thread`

```
● ● ●  
#include <iostream>  
#include <thread>  
  
void hello() {  
    std::cout << "Hello\n";  
}  
  
int main() {  
    std::thread t(hello); ←  
    t.join();  
}
```

Start a thread

The new thread starts **at once** and runs `hello()`. `main` does not wait: it goes on to the next line.

Now **two** threads run: `main` and `t`.

Threads in C++: join

```
● ● ●  
  
#include <iostream>  
#include <thread>  
  
void hello() {  
    std::cout << "Hello\n";  
}  
  
int main() {  
    std::thread t(hello);  
    t.join();  
}
```

Wait for the thread

main stops here until t has finished hello(). After join(), main can safely read everything that thread t wrote.

Rule: **join** every thread you start, and read its results only **after** join().

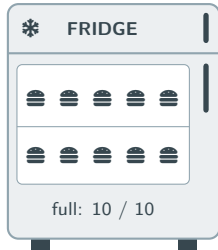
Threads in C++: an argument

```
● ● ●  
  
#include <iostream>  
#include <thread>  
  
void hello(const char *name) {  
    std::cout << name << " says hello\n";  
}  
  
int main() {  
    std::thread a(hello, "A");  
    std::thread b(hello, "B");  
    a.join();  
    b.join();  
}
```

Give the thread an argument

After the function, write its arguments.
Thread a runs hello("A").

Task 1: the office fridge



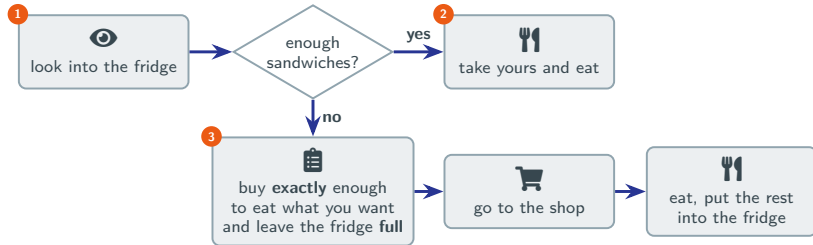
worker 0 wants 3

worker 1 wants 4

worker 2 wants 5

Each worker eats 20 times.

The rule for every worker



Example of step 3

The fridge has 1, worker 2 wants 5. Buy $10 - 1 + 5 = 14$, eat 5, put 9 in: $1 + 9 = 10$, full again.

Task 1: what to do



```
#include "office.h"
int fridge = CAPACITY; // sandwiches inside

void worker(int want, int pause_ms) {
    // TODO 1
}

void cleaner() {
    // TODO 2
}

int main() {
    // TODO 3
}
```

office.h (we do not look inside)
 CAPACITY = 10, MEALS = 20
 go_to_shop(): takes 50 ms, pause(ms): waits

Task

Open fridge.cpp from [GitHub](#). Three workers share the office fridge. A cleaner checks that it is not overfull.

1. Write worker(want, pause_ms): MEALS times, follow the rule, then pause(pause_ms).
2. Write cleaner(): every 10 ms, for 4 s, look into the fridge. More than CAPACITY? Print the fridge is overfull: and how many.
3. In main, start 4 threads and join them: worker(3, 30), worker(4, 40), worker(5, 50), cleaner()
4. Is this program safe? Answer **before** you run it.
5. Run it 3 times. What did the cleaner print? If the fridge was overfull: why?

Solution: worker()

```
● ● ●  
void worker(int want, int pause_ms) {  
    for (int i = 0; i < MEALS; i++) {  
        if (fridge >= want) {  
            fridge = fridge - want;           // eat  
        } else {  
            int buy = CAPACITY - fridge + want;  
            go_to_shop();  
            fridge = fridge + buy - want;     // eat, rest in  
        }  
        pause(pause_ms);  
    }  
}
```

Solution: cleaner() and main()

```

● ● ●
void cleaner() {
    for (int i = 0; i < 400; i++) { // 4 s
        if (fridge > CAPACITY)
            std::cout << "the fridge is "
                        << "overfull: "
                        << fridge << "\n";
        pause(10);
    }
}

```

```

● ● ●
int main() {
    std::thread w0(worker, 3, 30);
    std::thread w1(worker, 4, 40);
    std::thread w2(worker, 5, 50);
    std::thread c(cleaner);
    w0.join(); w1.join(); w2.join(); c.join();
}

```

Solution: what did the cleaner print?



```
$ ./fridge
the fridge is overfull: 11
the fridge is overfull: 11
the fridge is overfull: 19
the fridge is overfull: 28
...                      (35 lines)
```

Questions 4 and 5

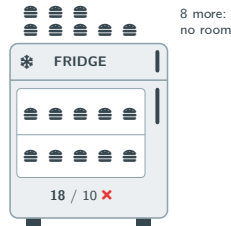
No. The cleaner found the fridge overfull in **every** run, with up to 28 sandwiches. The numbers are different every time.

Is the rule wrong?

No. If the workers eat **one after another**, the fridge always ends with exactly 10 ($\text{fridge} + \text{buy} - \text{want} = 10$). The problem appears only when they run **at the same time**.

Solution: why was the fridge overfull?

step	worker 0 (wants 3)	worker 1 (wants 4)	fridge
1	$2 \geq 3$? no		2
2	list: buy $10 - 2 + 3 = 11$		2
3	goes to the shop ...	$2 \geq 4$? no	2
4	...	list: buy $10 - 2 + 4 = 12$	2
5	eats 3, puts 8 in	goes to the shop ...	10
6		eats 4, puts 8 in	18 ✗



What went wrong

Both looked into the fridge **before** the other came back. Both bought for a fridge with 2 sandwiches. Capacity 10, but 18 inside.

The result depends on **when** the threads run. This has a name: a **race condition**.

Race condition

A **race condition**: “the results depend on the timing of the code’s execution.”

R. H. Arpaci-Dusseau, A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*

In the fridge: the result depended on **when** the workers looked inside.

First try: look again after the shop

Task 1: list, then shop

```

● ● ●
} else {
    int buy = CAPACITY - fridge + want;
    go_to_shop();
    fridge = fridge + buy - want;
}

```

First try: shop, then look again

```

● ● ●
} else {
    go_to_shop();
    int buy = CAPACITY - fridge + want;
    fridge = fridge + buy - want;
}

```



The gap between reading and writing fridge is now much **shorter**. Is the bug gone?

First try: is the bug gone?

	shop	pause after a meal	meals per worker	runs	overfull
as in Task 1	50 ms	30, 40, 50 ms	20	10	never
stress test	0 ms	none	1 000 000	3	36, 194, 228 times

Why so rare?

The gap is now between reading fridge for the list and writing it: less than a microsecond. With pauses, two workers almost never meet in it. Without pauses and with a million meals, they do.

The gap is **shorter**, not gone. A rare bug is still a bug.

Second try: a note on the fridge

```
● ● ●
bool note = false;    // "I am going shopping"

} else if (!note) {    // nobody is shopping
    note = true;        // leave the note
    int buy = CAPACITY - fridge + want;
    go_to_shop();
    fridge = fridge + buy - want;
    note = false;       // take the note away
                        // a note? come back later
}
```



Before you go shopping, leave a note. A worker who sees the note skips this meal.

Second try: does the note help?

step	worker 0	worker 1
1	no note? yes	
2		no note? yes
3	leaves the note	
4		leaves the note
5	goes to the shop	goes to the shop ✗

Both saw no note

Each one looked **before** the other left the note. Both go to the shop.

	shop	pause after a meal	meals per worker	runs	overfull
as in Task 1	50 ms	30, 40, 50 ms	20	10	never
stress test	0 ms	none	1 000 000	3	1 047, 1 943, 798 times

Checking the note is again “check, then act”: the same gap, only shorter.

What do we really need?



The idea

Lock the fridge **before** you look inside. Unlock it when you are done.
Fridge locked? **Wait** until it is unlocked.

A correct lock is hard to build yourself. C++ has one ready: `std::mutex`.

A lock in C++: `std::mutex`

```
● ● ●  
#include <mutex>  
  
std::mutex m;  
  
void work() {  
    // ... anything ...  
    m.lock();  
    // here only one thread at a time  
    m.unlock();  
    // ... anything ...  
}
```

The whole program

Many threads may run `work()`. Between `lock()` and `unlock()` there is only **one** of them at a time.

Three new things

`std::mutex`, `lock()` and `unlock()`. One by one.

A lock in C++: `std::mutex`

```
● ● ●  
#include <mutex>  
  
std::mutex m;  
  
void work() {  
    // ... anything ...  
    m.lock();  
    // here only one thread at a time  
    m.unlock();  
    // ... anything ...  
}
```

One lock

A global variable: all threads use the **same** lock. At the start it is unlocked.

A lock in C++: lock()

```
● ● ●  
#include <mutex>  
  
std::mutex m;  
  
void work() {  
    // ... anything ...  
    m.lock();  
    // here only one thread at a time  
    m.unlock();  
    // ... anything ...  
}
```

Take the lock

Unlocked? Take it and go on.
Locked by another thread? **Wait** here until it is unlocked.

A lock in C++: unlock()

```
● ● ●  
#include <mutex>  
  
std::mutex m;  
  
void work() {  
    // ... anything ...  
    m.lock();  
    // here only one thread at a time  
    m.unlock();  
    // ... anything ...  
}
```

Give the lock back

One thread that waits at lock() can now take it and go on.

Task 2: lock the fridge



```
void worker(int want, int pause_ms) {
    for (int i = 0; i < MEALS; i++) {
        if (fridge >= want) {
            fridge = fridge - want;
        } else {
            int buy = CAPACITY - fridge + want;
            go_to_shop();
            fridge = fridge + buy - want;
        }
        pause(pause_ms);
    }
}
```

Task

Take your program from Task 1, or the template from [GitHub](#).

1. Add a mutex, so that the fridge is **never** overfull: the cleaner must stay quiet.
2. Run it 3 times. What does the cleaner print now?
3. The cleaner only **reads** the fridge. Does it need the lock too?

Solution: lock the fridge



```
void worker(int want, int pause_ms) {
    for (int i = 0; i < MEALS; i++) {
        m.lock();
        if (fridge >= want) {
            fridge = fridge - want;
        } else {
            int buy = CAPACITY - fridge + want;
            go_to_shop();
            fridge = fridge + buy - want;
        }
        m.unlock();
        pause(pause_ms);
    }
}
```

Question 1

Lock **before** looking into the fridge, unlock **after** the last change. The whole rule is one step now.

Question 2

Nothing. The fridge is never overfull. (Without the lock: 30 to 50 complaints.)

The lock covers the whole rule: look, decide, change.
The price: while one worker is in the shop, the others wait at `m.lock()`.

Solution: does the cleaner need the lock?

```
● ● ●  
void cleaner() {  
    for (int i = 0; i < 400; i++) {  
        m.lock();  
        if (fridge > CAPACITY)  
            std::cout << "...";  
        m.unlock();  
        pause(10);  
    }  
}
```

Question 3

Yes. A worker may write `fridge` while the cleaner reads it: one writer is enough for a race (in C++: a **data race**).

Every access to shared data, **read or write**, goes under the lock.

Every lock needs an unlock



```
for (int i = 0; i < MEALS; i++) {
    m.lock();
    if (fridge == 0) {           // empty? I go home!
        std::cout << "a worker goes home\n";
        return;                // ... with the key
    }
    ...
    m.unlock();
    pause(pause_ms);
}
```



```
$ ./fridge
a worker goes home
    (nothing more: the program hangs)
```

What happened?

The worker locked the fridge and went home. Nobody unlocks it: the others wait at `m.lock()` **forever**.

Every `lock()` needs an `unlock()`, on **every** way out of the code.

Good to know: `std::lock_guard`

```

● ● ●
for (int i = 0; i < MEALS; i++) {
    std::lock_guard<std::mutex> g(m); // lock
    if (fridge == 0) {
        return;           // unlocks by itself
    }
    ...
}                          // unlocks by itself
pause(pause_ms);
}

```

How does it work?

`lock_guard` locks the mutex when it is created. At the end of its block, at `}` or at `return`, it unlocks the mutex **by itself**. You cannot forget `unlock()`.

Good to know: in C++ code you will usually see `lock_guard`, not `lock()` and `unlock()`.

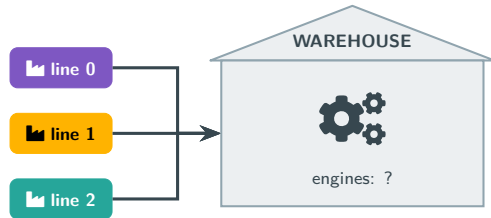
The engine factory

```
● ● ●
#include <iostream>
#include <thread>

int engines = 0;           // engines in the warehouse

void line() {
    for (int i = 0; i < 1000000; i++)
        engines++;         // one more engine
}

int main() {
    std::thread t0(line), t1(line), t2(line);
    t0.join(); t1.join(); t2.join();
    std::cout << "in stock: " << engines << "\n";
}
```



Each line builds **1 000 000** engines.
After each one: `engines++`.

3 lines × 1 000 000 engines. How many engines are in stock at the end?

Try it: does it really lose engines?



```
$ g++ -O0 -pthread factory.cpp -o factory
$ ./factory
in stock: 1105383
$ ./factory
in stock: 1236626
$ ./factory
in stock: 1141892
```

Try it

The code is on [GitHub](#). Compile with `-O0` and run it a few times. 3 000 000 engines built, about **1.1 to 1.4 million** in stock. A different number every time.

With `-O2` the compiler makes the loop one big addition and the bug hides. **Hidden is not gone.**

Why? engines++ is read, add, write



engines++;



```
la    a0, engines    # a0 = address of engines
lw    a1, 0(a0)      # read engines
addi  a1, a1, 1       # add 1
sw    a1, 0(a0)      # write engines
```

RISC-V, clang 18 -O2

step	line 0	line 1	engines
1	lw: a1 = 100		100
2		lw: a1 = 100	100
3	addi: a1 = 101		100
4		addi: a1 = 101	100
5	sw: engines = 101		101
6		sw: engines = 101	101 ✗

Two engines built, one counted. Like the fridge: another thread came in **between** read and write.

We can still fix it with a mutex



```
std::mutex m;  
  
void line() {  
    for (int i = 0; i < 1000000; i++) {  
        m.lock();  
        engines++;  
        m.unlock();  
    }  
}
```



```
$ ./factory  
in stock: 3000000  
$ ./factory  
in stock: 3000000  
$ ./factory  
in stock: 3000000
```

It works: every engine is counted.
But it is expensive: a whole lock to protect a single ++.

What does a mutex do?

1 The lock is free: short



2 The lock is taken: long



A free lock: a quick step. A taken lock: two calls to the operating system, and they cost far more than one ++. For one variable, C++ has something simpler.

For one variable: `std::atomic`



```
#include <atomic>
#include <iostream>
#include <thread>

std::atomic<int> engines = 0;

void line() {
    for (int i = 0; i < 1000000; i++)
        engines++;
}

int main() {
    std::thread t0(line), t1(line), t2(line);
    t0.join(); t1.join(); t2.join();
    std::cout << "in stock: " << engines << "\n";
}
```

The whole program

The engine factory again, without a mutex. Only one line is different.

Two new things

`std::atomic<int>` and what `engines++` does now. One by one.

For one variable: `std::atomic<int>`

```
● ● ●  
#include <atomic>  
#include <iostream>  
#include <thread>  
  
std::atomic<int> engines = 0;  
  
void line() {  
    for (int i = 0; i < 1000000; i++)  
        engines++;  
}  
  
int main() {  
    std::thread t0(line), t1(line), t2(line);  
    t0.join(); t1.join(); t2.join();  
    std::cout << "in stock: " << engines << "\n";  
}
```

An atomic variable

Like an `int`, but every operation on it is **one step** that cannot be split.

For one variable: engines++

```

● ● ●
#include <atomic>
#include <iostream>
#include <thread>

std::atomic<int> engines = 0;

void line() {
    for (int i = 0; i < 1000000; i++)
        engines++;
}

int main() {
    std::thread t0(line), t1(line), t2(line);
    t0.join(); t1.join(); t2.join();
    std::cout << "in stock: " << engines << "\n";
}

```

One step

Read, add 1 and write **at once**: no other thread can come in between.

On RISC-V it is one instruction: `amoadd.w`. No lock, no operating system.

Mutex or atomic: which one is faster?



```
$ time ./factory          # mutex
in stock: 30000000
real    0m2.131s
user    0m2.588s
sys     0m2.942s
```



```
$ time ./factory          # atomic
in stock: 30000000
real    0m0.617s
user    0m1.809s
sys     0m0.003s
```

The same factory, **10 000 000** engines per line. Run with time:

real: time on the clock

The atomic is about **3 times** faster.

sys: time in the operating system

The mutex spends about 3s there: threads go to sleep and wake up again and again. The atomic never asks the operating system: sys is almost 0.

One operation on one variable? Use `std::atomic`.

An atomic is one operation, not more



```
std::atomic<int> counter = 0;

counter = 5;           // atomic write
                       // <- another thread can run here
int x = counter;       // atomic read: is x 5? not sure
                       // <- another thread can run here
counter = x + 1;       // atomic write
counter++;             // one atomic operation
```

Each operation is atomic

Nobody can come in **during** one operation.

Between two operations

Another thread can run, or the operating system can stop this thread for a while.

`std::atomic` protects **one** operation, not a group of them.

counter = counter + 1: two operations



```
counter = counter + 1;  // atomic read,
                        // then atomic write:
                        // a gap inside one line!
```

step	line 0	line 1	counter
1	read: 5		5
2		read: 5	5
3	write 6		6
4		write 6	6 ✗

Measured

3 threads × 1 000 000 times counter = counter + 1:
counter is **1.2 to 1.7 million**.

With counter++

Always 3 000 000: read, add and write are **one** operation.

One operation on one variable: **atomic**. Several operations that belong together: **mutex**.

Task 3: an order for 1000 engines



```
#include <atomic>
#include "factory.h"

int built_by[3];    // engines built by each line

void line(int id, int build_ms) {
    // TODO
}

int main() {
    std::thread t0(line, 0, 1);    // fast
    std::thread t1(line, 1, 2);
    std::thread t2(line, 2, 3);    // slow
    t0.join(); t1.join(); t2.join();
    check_order();
    // prints built_by[0], [1], [2]
}
```

Task

Open `factory.cpp` from [GitHub](#).

1. A customer orders **exactly** `ORDER` engines: not one more, not one less.
2. Every engine gets its own serial number: 1, 2, ..., 1000. No number twice.
3. Use `std::atomic`, **no mutex**. Run it 3 times: is the order OK?
4. Count in `built_by[id]` how many engines each line built. Why are they different?

`factory.h` (we do not look inside)

`ORDER = 1000`

`build_engine(serial, ms)`: builds one engine

`check_order()`: is the order OK?

Solution: take a number first



```
std::atomic<int> next = 1;
void line(int id, int build_ms) {
    while (true) {
        int serial = next++;    // take a number
        if (serial > ORDER)    // order done?
            break;
        build_engine(serial, build_ms);
        built_by[id]++;
    }
}
```

Questions 1 to 3

next++ reads and adds in one step and returns the **old** value: no two calls get the same number.



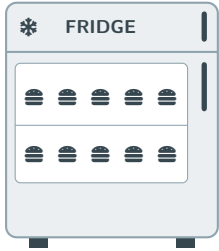
```
$ ./factory
engines built: 1000 (order: 1000)
serial numbers used twice: 0,
    missing: 0, above 1000: 0
order OK
line 0 built 527 engines
line 1 built 281 engines
line 2 built 192 engines
```

Question 4

Nobody divided the work: the fast line comes back for a number more often.

Take a number first, then work: **one atomic operation** decides everything.

Two locks



fridge_m



micro_m

```
● ● ●
std::mutex fridge_m;    // the fridge
std::mutex micro_m;    // the microwave
```

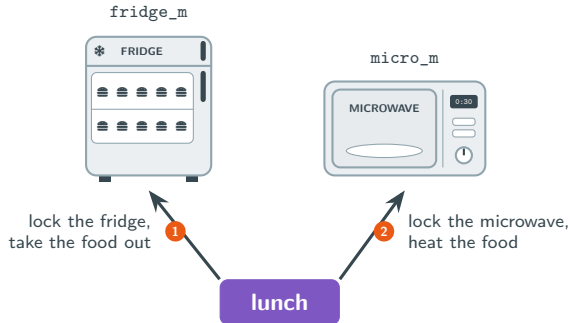
New in the office: a microwave
Only one person at a time may use it, so it gets its **own** lock.

Some workers need **both**: the fridge and the microwave.

Two workers: lunch



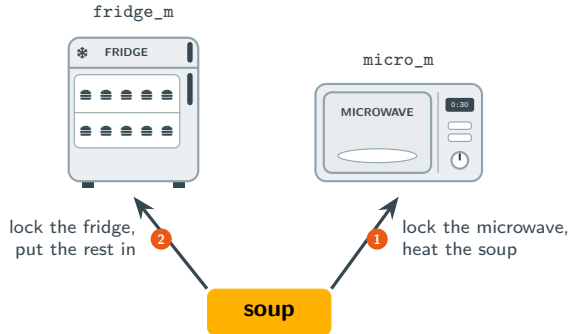
```
void lunch() {    // take it out, then heat it
    for (int i = 0; i < MEALS; i++) {
        fridge_m.lock();    // 1. fridge
        take_out();
        micro_m.lock();    // 2. microwave
        heat();
        micro_m.unlock();
        fridge_m.unlock();
        pause(30);
    }
}
```



lunch: first the **fridge**, then the **microwave**. It keeps the fridge while it heats.

Two workers: soup

```
void soup() {  
    // heat it, put the rest back  
    for (int i = 0; i < MEALS; i++) {  
        micro_m.lock();           // 1. microwave  
        heat();  
        fridge_m.lock();         // 2. fridge  
        put_in();  
        fridge_m.unlock();  
        micro_m.unlock();  
        pause(40);  
    }  
}
```



soup: first the **microwave**, then the **fridge**: the other order.
Each function alone is correct: every lock() has its unlock().

Try it: does it finish?

```
● ● ●  
$ ./deadlock  
    (nothing: the program hangs)  
^C  
$ ./deadlock  
    (nothing: the program hangs)  
^C
```

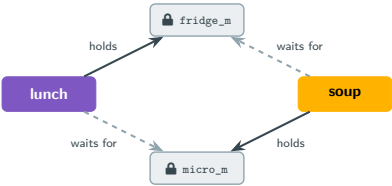
Try it

The code is on [GitHub](#). Run it a few times. It often **hangs** and never prints all done.

No error, no crash, no output: the program just stands still.

Why does it hang?

step	lunch	soup
1	fridge_m.lock() ✓ got it	
2		micro_m.lock() ✓ got it
3	micro_m.lock() ... wait	
4		fridge_m.lock() ... wait

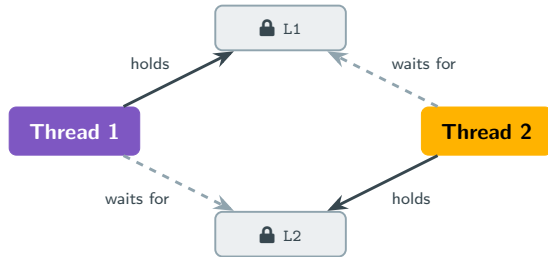


lunch waits for soup, soup waits for lunch.
 Each one holds what the other needs. Nobody gives a lock back: they wait **forever**.

Deadlock

“**Deadlock** occurs, for example, when a thread (say Thread 1) is holding a lock (L1) and waiting for another one (L2); unfortunately, the thread (Thread 2) that holds lock L2 is waiting for L1 to be released.”

R. H. Arpaci-Dusseau, A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*



In the office: Thread 1 = lunch, L1 = fridge_m

Thread 2 = soup, L2 = micro_m

The slide “Every lock needs an unlock” also hung, but it was different. There, one worker forgot `unlock()`: one function had a bug. Deadlock: each function alone is correct. The bug is in **how they meet**.

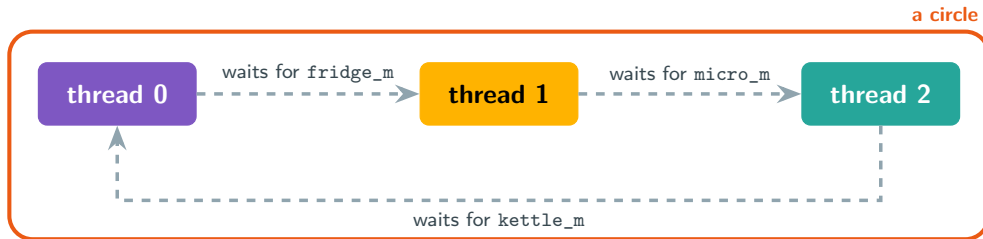
Who waits for whom: a chain



An arrow: “waits for a lock that the other thread holds”
Thread 2 waits for nobody: it finishes and unlocks. Then thread 1 can go on, then thread 0.

A chain ends somewhere: everybody gets their turn. Slow, but no deadlock.

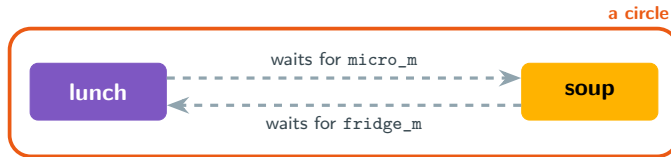
Who waits for whom: a circle



The last one waits for the first one
Nobody can finish, so nobody unlocks.

Thread 0 waits for 1, 1 for 2, ..., the last one for thread 0: a **circle**. This is a deadlock.

lunch and soup: the smallest circle



Two threads are enough

lunch waits for soup, soup waits for lunch.

A deadlock needs a circle. No circle, no deadlock. In **Fixing a deadlock**: break the circle.

Task 4: who waits for whom?



```
std::mutex fridge_m, micro_m, kettle_m;
void worker(std::mutex &first, std::mutex &second,
            int pause_ms) {
    for (int i = 0; i < MEALS; i++) {
        first.lock(); use();
        second.lock(); use();
        second.unlock();
        first.unlock();
        pause(pause_ms);
    }
}
// std::ref: the thread gets the mutex itself, not a copy
std::thread w0(worker, std::ref(fridge_m), std::ref(micro_m), 30);
std::thread w1(worker, std::ref(micro_m), std::ref(kettle_m), 40);
// office A:
std::thread w2(worker, std::ref(kettle_m), std::ref(fridge_m), 50);
// office B:
std::thread w2(worker, std::ref(fridge_m), std::ref(kettle_m), 50);
```

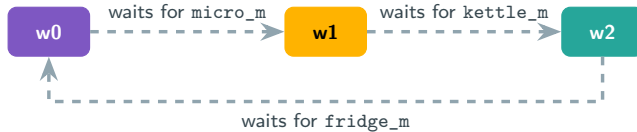
Task

Open office3.cpp from [GitHub](#). Office A and office B differ only in worker 2.

1. For each office, draw who can wait for whom.
2. Which office can hang? Answer **before** you run it.
3. Run each office 3 times. Were you right?
4. Office A with only two of the three workers: can it hang?

Solution: who waits for whom?

Office A: 10 of 10 runs hang



Questions 1 to 3

A: a **circle**, so it can hang.

B: everyone goes left to right in the **same order**: no circle.

Question 4

No: two workers of office A never make a circle.

Office B: 0 of 10 runs hang

order: fridge_m → micro_m → kettle_m



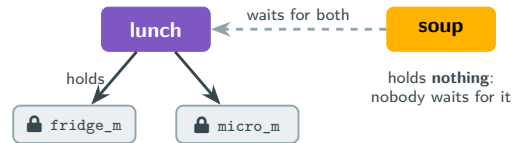
A deadlock needs a circle. No circle, no deadlock.

How to break the circle?



Fix 1: one order

Everybody takes the locks in the **same order**. The arrows always point the same way, so the circle can never close.



Fix 2: `scoped_lock`

Take both locks **at once**. A thread that waits holds **nothing**, so nobody waits for it: no arrow can end at it.

Office B in Task 4 already used Fix 1.

Fix 1: one order for everyone

Before

```

● ● ●
void soup() {
    for (int i = 0; i < MEALS; i++) {
        micro_m.lock();    // 1. microwave
        heat();
        fridge_m.lock();   // 2. fridge
        put_in();
        fridge_m.unlock();
        micro_m.unlock();
        pause(40);
    }
}

```

After

```

● ● ●
void soup() {
    for (int i = 0; i < MEALS; i++) {
        fridge_m.lock();   // 1. fridge
        micro_m.lock();    // 2. microwave
        heat();
        put_in();
        micro_m.unlock();
        fridge_m.unlock();
        pause(40);
    }
}

```

The rule for everyone: first the **fridge**, then the **microwave**.
lunch() already keeps to it. Only soup() changes.

Fix 1: why does it work?

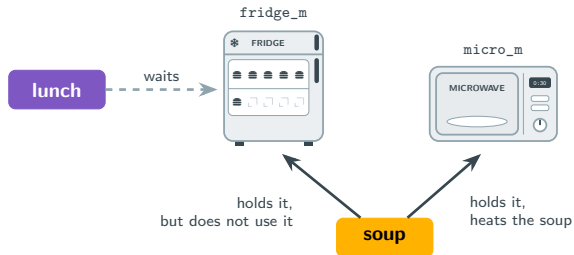
step	lunch	soup
1	fridge_m.lock() ✓ got it	
2		fridge_m.lock() ... wait, holds nothing
3	micro_m.lock() ✓ got it	
4	heat(), unlock both	
5		fridge_m ✓, micro_m ✓

Whoever gets fridge_m first gets everything

The other one waits, but holds **nothing**: nobody waits for it.

The circle can never close. 0 of 10 runs hang.

Fix 1: the price



lunch waits longer than needed

soup holds the fridge while it heats the soup. lunch must wait for the fridge, although soup does not use it yet.

The rule lives only in our heads

The compiler does not check the order. One function with the wrong order, anywhere in the program: deadlock again.

A fixed order works, but **everybody** must know it and keep to it.

Fix 2: `std::scoped_lock`

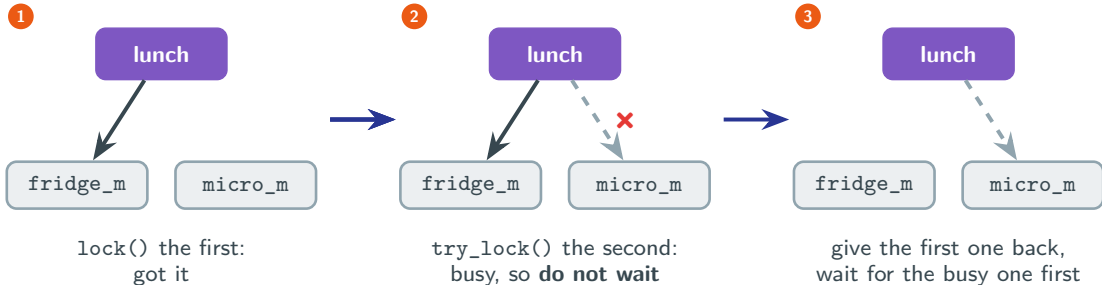
```
void soup() {  
    for (int i = 0; i < MEALS; i++) {  
        {  
            std::scoped_lock both(micro_m, fridge_m);  
            heat();  
            put_in();  
        }  
        pause(40);  
    }  
}
```

Take both locks

One call: **both** locks or **none**. The order in () does not matter. At } both are unlocked, like `lock_guard`.

`lunch()` does the same: `std::scoped_lock both(fridge_m, micro_m);`

How does `scoped_lock` do it?



No circle

A thread never holds one lock while it waits for another, so nobody waits for it.

Good to know: in C++ code you will usually see `scoped_lock` when two locks are needed.

Task 5: fix office A



```
void worker(std::mutex &first, std::mutex &second,
            int pause_ms) {
    for (int i = 0; i < MEALS; i++) {
        first.lock();    use();
        second.lock();   use();
        second.unlock();
        first.unlock();
        pause(pause_ms);
    }
}

std::thread w0(worker, std::ref(fridge_m), std::ref(micro_m), 30);
std::thread w1(worker, std::ref(micro_m), std::ref(kettle_m), 40);
std::thread w2(worker, std::ref(kettle_m), std::ref(fridge_m), 50);
```

Task

Take office A from Task 4, or the template from [GitHub](#).

1. Fix it with **one order**: change only main().
2. Fix it with `scoped_lock`: change only worker().
3. Run each fix 3 times. Does it hang?
4. Which fix works without knowing the other workers?

Solution: the code

Fix 1: one order (fridge, microwave, kettle). Only worker 2 breaks it:



```
std::thread w2(worker, std::ref(fridge_m), std::ref(kettle_m), 50); // was: kettle_m, fridge_m
```

Fix 2: `scoped_lock`. `main()` stays as in office A:



```
void worker(std::mutex &first, std::mutex &second,
            int pause_ms) {
    for (int i = 0; i < MEALS; i++) {
        {
            std::scoped_lock both(first, second);
            use();
            use();
        }
        pause(pause_ms);
    }
}
```

Solution: which fix to use?

	runs	hang	time
no fix	10	10	hangs
one order	10	0	1.06 s
scoped_lock	10	0	1.06 s

Questions 1 to 3

Both fixes work: 0 of 10 runs hang.

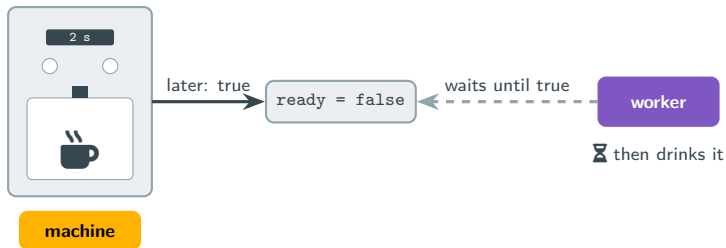
Question 4

scoped_lock: each worker fixes **itself**.

One order: **all** workers must agree on it.

Locks you need at the same time: `scoped_lock`.
 Locks you take at different times: one fixed order.

Waiting for the coffee



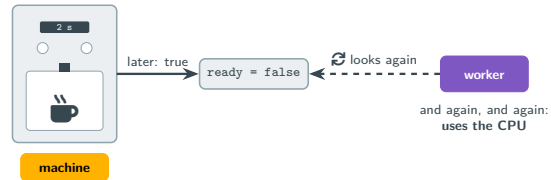
```
bool ready = false;  // is the coffee ready?
```

Two threads: the machine makes a coffee (2s), the worker must wait until it is ready.
How should the worker wait?

First try: look again and again

```

void worker() {
    m.lock();
    while (!ready) {           // not ready?
        m.unlock();           // let the machine in
        m.lock();              // look again
    }
    m.unlock();
    drink_coffee();
}
    
```



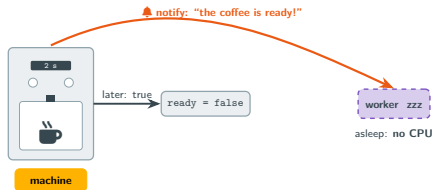
Busy waiting

The worker waits 2s and uses the CPU **all that time**.
time ./coffee: real 2.0s, user 2.0s. A whole core does nothing useful.

What we need: “put me to **sleep** until somebody calls me”.

Sleep until somebody calls you

A **condition variable** (`std::condition_variable`) lets a thread **sleep** until another thread tells it: something has changed.



`cv.wait(...)`: the worker

Sleep, without the CPU, until somebody calls `notify`.

`cv.notify_one()`: the machine

Wake up one thread that sleeps in wait.

The coffee with a condition_variable

```

● ● ●
std::mutex m;
std::condition_variable cv;
bool ready = false;

void machine() {
    make_coffee(2000);           // 2 s
    std::lock_guard<std::mutex> g(m);
    ready = true;
    cv.notify_one();
}

void worker() {
    std::unique_lock<std::mutex> lk(m);
    while (!ready)
        cv.wait(lk);
    drink_coffee();
}

```

The whole program

The machine makes the coffee and calls the worker. The worker sleeps until then.

Four new things

unique_lock, wait(), while and notify_one().
One by one.

condition_variable: unique_lock

```

● ● ●
std::mutex m;
std::condition_variable cv;
bool ready = false;

void machine() {
    make_coffee(2000);           // 2 s
    std::lock_guard<std::mutex> g(m);
    ready = true;
    cv.notify_one();
}

void worker() {
    std::unique_lock<std::mutex> lk(m);
    while (!ready)
        cv.wait(lk);
    drink_coffee();
}

```

Lock m

Like lock_guard: locks now, unlocks at the end of the block. But wait() can also unlock it and lock it again.

condition_variable: wait()

```
● ● ●
std::mutex m;
std::condition_variable cv;
bool ready = false;

void machine() {
    make_coffee(2000);           // 2 s
    std::lock_guard<std::mutex> g(m);
    ready = true;
    cv.notify_one();
}

void worker() {
    std::unique_lock<std::mutex> lk(m);
    while (!ready)
        cv.wait(lk);
    drink_coffee();
}
```

Sleep until somebody calls you

- 1 unlock `m` (the machine can come in)
- 2 sleep: **no CPU** at all
- 3 woken up? lock `m` again, go on

condition_variable: while

```

● ● ●
std::mutex m;
std::condition_variable cv;
bool ready = false;

void machine() {
    make_coffee(2000);           // 2 s
    std::lock_guard<std::mutex> g(m);
    ready = true;
    cv.notify_one();
}

void worker() {
    std::unique_lock<std::mutex> lk(m);
    while (!ready)
        cv.wait(lk);
    drink_coffee();
}

```

Why while?

Look first: is the coffee already ready?
Then do not sleep at all.

After waking up, **look again**. (A thread can even wake up without any reason.)

condition_variable: notify_one()

```

● ● ●
std::mutex m;
std::condition_variable cv;
bool ready = false;

void machine() {
    make_coffee(2000);           // 2 s
    std::lock_guard<std::mutex> g(m);
    ready = true;
    cv.notify_one();
}

void worker() {
    std::unique_lock<std::mutex> lk(m);
    while (!ready)
        cv.wait(lk);
    drink_coffee();
}

```

Call the worker

ready is true: wake up **one** thread that sleeps in `cv.wait()`.

Several threads wait? `cv.notify_all()` wakes **all** of them.

Try it: does the worker still use the CPU?



```
$ time ./coffee      # busy
real    0m2.002s
user    0m1.964s
sys     0m0.004s
```



```
$ time ./coffee_cv  # cv
real    0m2.002s
user    0m0.001s
sys     0m0.001s
```

Try it

The code is on [GitHub](#). Run both versions with time.

The same real time

The coffee takes 2s in both versions.

Almost no user time

With the condition_variable, the worker sleeps: no CPU.

Waiting with a condition_variable costs (almost) nothing.

Task 6: the lunch bell



```
bool bell = false;    // has the bell rung?
std::mutex m;
void boss() {
    pause(2000);      // lunch time in 2 s
    m.lock();
    bell = true;      // ring the bell
    m.unlock();
}
void worker(int id) {
    m.lock();
    while (!bell) {   // no bell yet?
        m.unlock();
        m.lock();
    }
    m.unlock();
    go_to_lunch(id);
}
```

Task

Open `bell.cpp` from [GitHub](#). Three workers wait for the lunch bell. It waits in a loop, like the first coffee.

1. Run it with `time`. Write down `user` and `sys`.
2. Replace busy waiting with a `condition_variable`. The boss wakes everybody up with `notify_all()`. Run it with `time` again.
3. Change `notify_all()` to `notify_one()`. What happens? Why?

If you finish early: add `pause(3000);` at the start of `worker()`. The bell rings before anybody waits. Do they hang? Why not?

Solution: the lunch bell



```
std::condition_variable cv;

void boss() {
    pause(2000);
    std::lock_guard<std::mutex> g(m);
    bell = true;
    cv.notify_all();    // wake up all
}

void worker(int id) {
    std::unique_lock<std::mutex> lk(m);
    while (!bell)
        cv.wait(lk);
    go_to_lunch(id);
}
```



```
$ ./bell
worker 0 goes to lunch
worker 1 goes to lunch
worker 2 goes to lunch
```

The same pattern as the coffee

The boss: change bell, then notify. The workers: while not rung, wait.

Solution: the questions

	real	user	sys
busy waiting	2.0 s	2.5 s	3.0 s
condition_variable	2.0 s	0.001 s	0.004 s

Questions 1 and 2

Three workers wait 2s and use **more than 5 s** of CPU.
 With the condition_variable: almost nothing.

Question 3: notify_one()

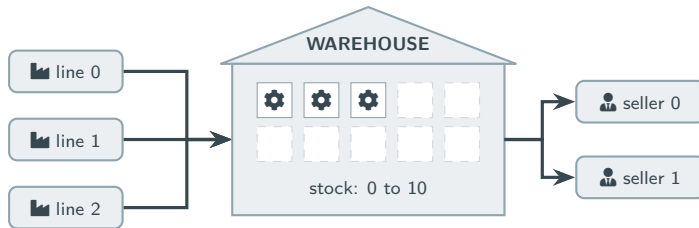
One worker goes to lunch, then the program **hangs**.
 notify_one() wakes only one. The other two sleep forever.

If you finished early

It does not hang. Nobody waited when the bell rang, but bell is already true: the while does not call wait() at all.

Change the condition, then notify. Wait in a while. Many waiting: notify_all().

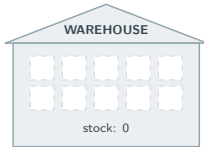
The factory sells engines



```
const int CAPACITY = 10;    // room in the warehouse
int stock = 0;              // engines in the warehouse
```

3 lines build 100 engines each. 2 sellers sell 150 engines each.
All of them share one warehouse with room for **10** engines.

Two kinds of waiting



The warehouse is empty

A seller has nothing to sell. It must wait until a line puts an engine in.



The warehouse is full

A line has no room for its engine. It must wait until a seller takes an engine out.

The producer-consumer problem

Producers (lines) put things in, consumers (sellers) take them out. The buffer (warehouse) between them is not endless.

A lock alone is not enough: we must **wait until something changes**.

The warehouse: one condition_variable, two kinds of waiting



```
void seller() {
    for (int i = 0; i < SALES; i++) {
        std::unique_lock<std::mutex> lk(m);
        while (stock == 0) // empty?
            cv.wait(lk);
        stock--;
        check();
        cv.notify_all();
    }
    sell_engine();
}
```

Like the coffee

The seller sleeps in `wait()` while the warehouse is empty.

New: two kinds of waiting

Sellers wait for engines, lines wait for room. Both sleep on the **same** cv.

So `notify_all()`

After every change, wake everybody up: each one looks again in its own while.

`check()`

A ready function: is stock still between 0 and 10?

Task 7: the warehouse

Task

Open `warehouse.cpp` from [GitHub](#). It waits in a loop, like the first coffee. Lines and sellers both wait there.

1. Run it with time. Write down user and sys.
2. Replace busy waiting with a `condition_variable`, in `seller()` and in `line()`. Run it with time.
3. Is it still correct? The program prints: built, sold, and how many times the stock was below 0 or above 10.
4. Change `while` to `if`. Run it 3 times. What happens? Why?
5. Remove `notify_all()`. What happens? Why?

```
● ● ●
$ ./warehouse
built: 300, sold: 300, in stock: 0,
stock below 0 or above 10: 0 times
```

Solution: line() and seller()

```

● ● ●
void line() {
    for (int i = 0; i < ENGINES; i++) {
        build_engine();
        std::unique_lock<std::mutex> lk(m);
        while (stock == CAPACITY)    // full?
            cv.wait(lk);
        stock++;
        check();
        cv.notify_all();
    }
}

```

```

● ● ●
void seller() {
    for (int i = 0; i < SALES; i++) {
        {
            std::unique_lock<std::mutex> lk(m);
            while (stock == 0)        // empty?
                cv.wait(lk);
            stock--;
            check();
            cv.notify_all();
        }
        sell_engine();
    }
}

```

One condition_variable for both: lines wait for room, sellers wait for engines.
notify_all() wakes both kinds, and each one looks again in its own while.

Solution: busy waiting or condition_variable?

```

● ● ●
$ time ./warehouse      # busy
built: 300, sold: 300 ...
real    0m0.766s
user    0m0.775s
sys     0m0.674s
    
```

```

● ● ●
$ time ./warehouse      # cv
built: 300, sold: 300 ...
real    0m0.781s
user    0m0.001s
sys     0m0.016s
    
```

Questions 1 to 3
 Both are correct: 300 built, 300 sold, stock always between 0 and 10. The same real time: the work is the same. But almost **no CPU**: the waiting threads sleep.

Waiting with condition_variable costs (almost) nothing.

Solution: if instead of while

step	who	what	stock
1	lines 0, 1, 2	stock == 10: all three wait	10
2	seller 0	stock--, notify_all(): all three wake up	9
3	line 0	stock++	10
4	line 1	if: does not look again, stock++	11 ✗
5	line 2	if: does not look again, stock++	12 ✗

Question 4

With if: the stock was below 0 or above 10 about **565 times** in every run. notify_all() wakes everybody, but only one of them may go on. The others must look again.

After wait(), always look again: while, not if.

Solution: without notify_all()



```
$ ./warehouse
      (nothing: the program hangs)

^C
```

Question 5

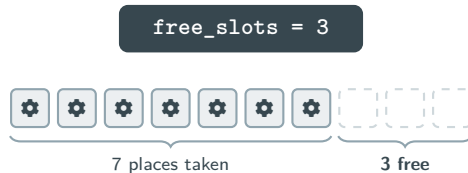
It **hangs** (3 of 3 runs). The sellers find the warehouse empty and sleep. The lines fill it up and sleep too. Nobody ever wakes anybody up.

Not a deadlock

Nobody holds a lock. Everybody sleeps and waits for a call that never comes.

Every change that someone may wait for needs a **notify**.

A semaphore



acquire(): take one

Counter > 0? Take one: counter - 1.

Counter = 0? **Wait** until it is not.

release(): give one back

Counter + 1.

Somebody waits? One of them goes on.

A **semaphore** is a counter of free things, with waiting built in.

Semaphores in C++



```
#include <semaphore>

std::counting_semaphore<10> free_slots{10};

free_slots.acquire(); // take one: -1, or wait at 0
// ...
free_slots.release(); // give one back: +1
```

<10> and {10}

<10>: the largest value

{10}: the value at the start

No while, no notify_all()

The waiting is inside acquire().

C++20: compile with `g++ -std=c++20`.

The warehouse with two semaphores

semaphore	start	counts	line	seller
free_slots	10	room for engines	acquire() before putting one in	release() after taking one out
engines	0	engines to sell	release() after putting one in	acquire() before taking one out

Full warehouse

free_slots is 0: the line waits in acquire().

Empty warehouse

engines is 0: the seller waits in acquire().

The warehouse: the code

```

● ● ●
void line() {
    for (int i = 0; i < ENGINES; i++) {
        build_engine();
        free_slots.acquire(); // wait for room
        {
            std::lock_guard<std::mutex> g(m);
            stock++;
            check();
        }
        engines.release(); // one more engine
    }
}

```

```

● ● ●
void seller() {
    for (int i = 0; i < SALES; i++) {
        engines.acquire(); // wait for one
        {
            std::lock_guard<std::mutex> g(m);
            stock--;
            check();
        }
        free_slots.release(); // one more room
        sell_engine();
    }
}

```

The semaphores do the waiting. The mutex still protects stock: two sellers can take an engine at the same time.
Correct in **20 of 20** runs: 300 built, 300 sold, stock always between 0 and 10.

condition_variable or semaphore?

	condition_variable	semaphore
waits for	any condition: <code>while (...)</code>	a counter above 0
wakes up	<code>notify_all()</code> : everybody looks again	<code>release()</code> : one waiting thread
a signal when nobody waits yet	is lost: that is why we look at <code>stock</code> first	is kept in the counter
CPU while waiting	none	none

Counting free things: a **semaphore**. Any other condition: **condition_variable**.

Good to know: a semaphore as a limit



```
std::counting_semaphore<2> microwaves{2}; // 2 of them

void lunch() {
    microwaves.acquire(); // at most 2 people heat
    heat();
    microwaves.release();
}
```

A mutex is almost a semaphore with a counter of 1

The difference: only the thread that locked a mutex may unlock it. A semaphore can be released by **any** thread: lines release engines, sellers acquire them.

A semaphore with N lets at most N threads in at the same time.